

ProjWT — Quickstart & System Overview

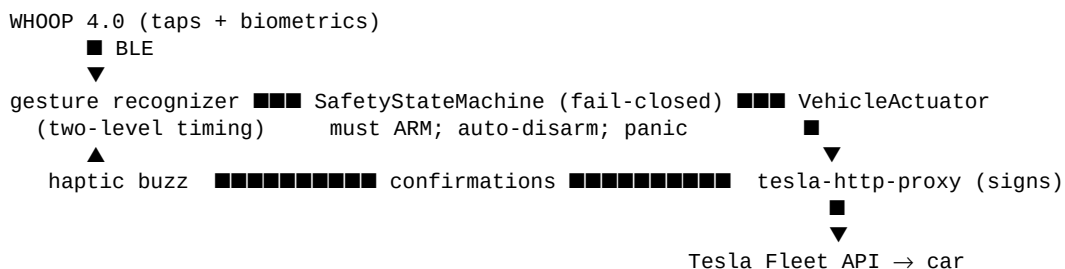
Private portfolio project. Gesture control of a real Tesla from a reverse-engineered WHOOP 4.0 band, with a fail-closed safety layer and haptic feedback. This doc compiles everything that works end to end and how to run it.

What works (end to end, on real hardware)

- **WHOOP 4.0 reverse-engineered over BLE** — custom GATT protocol, framing (CRC8 + zlib-CRC32) independently verified, pairing, Windows link stability.
- **Live biometrics** — realtime HR/RR decode (validated against actual BPM).
- **Raw banked sensor data** — HR, R-R, gravity/orientation vector, SpO2, skin-temp, respiration, pulled via the historical-offload path and decoded to CSV + an offline HTML dashboard.
- **Tap/flick gesture recognition** — two-level timing (collapses ringing into one tap, groups deliberate taps into a gesture), tunable, with live calibration.
- **Fail-closed safety state machine** — must ARM before any command; auto-disarm on inactivity; panic gesture locks + disarms.
- **Haptic feedback** — the band buzzes on connect / arm / confirm / panic, so you feel state without a screen.
- **Real Tesla control** — gesture → safety FSM → signed Vehicle Command Protocol (via tesla-http-proxy) → the car. **A tap-code unlocks/locks the real 2018 Model 3.**

Discrete, non-motion commands only (unlock, lock, flash, honk, frunk, vent, climate, charge). There is no third-party API for vehicle motion/Summon; the continuous-hold DRIVE demo is built against a simulator, not the real car.

The pipeline



Run it — ONE command

From the repo root, in PowerShell:

```
powershell -ExecutionPolicy Bypass -File launch_PWT.ps1
```

This refreshes the token, starts the signing proxy, waits for it, sets the env vars, and launches gesture control. Band control is **tap-to-cycle, pause-to-confirm** (robust to tap bounce — counting taps isn't):

- 1 **Tap the band once** (firm) to wake it — you feel a "ready" buzz; it arms and shows [UNLOCK].
- 1 **Each tap** advances the menu: UNLOCK → LOCK → FLASH → HONK → ... → CANCEL.
- 2 **Pause ~2.5s** on the command you want → it fires (confirm buzz). Then it disarms.
- 3 Cycle to [CANCEL] and pause to back out safely (does nothing).

So: unlock = one tap, wait. Lock = tap, tap, wait. Tune the pause with `--dwell N`. Edit the paths/VIN/band address at the top of `launch_PWT.ps1` if yours differ.

Run it — manually (two windows)

Window 1 — the signing proxy (keep open):

```
cd $env:USERPROFILE\vehicle-command
.\tesla-http-proxy.exe -tls-key C:\Users\thebl\tls-key.pem -cert C:\Users\thebl\tls-cert.pem -key-file C:
```

Window 2 — the controller:

```
cd "C:\Users\thebl\OneDrive\Documents\GitHub\projwt\src"
$env:TESLA_VEHICLE_TAG="5YJ3E1EA7JF029858" # VIN, NOT the Fleet API id
$env:TESLA_FLEET_BASE="https://localhost:4443" # the proxy
python gesture_control_PWT.py D1:86:73:D4:62:86 --backend tesla
```

Tuning the taps

Run the safe calibrator (no commands sent) and tap:

```
python gesture_control_PWT.py D1:86:73:D4:62:86 --calibrate
```

- Each tap prints its strength; `.weak NNNN` means it's below the threshold.
- Set `--strength` just under your comfortable light-tap number (default 2200).
- `--flick-gap` (0.30) = impulses within this window are the same tap (debounce).
- `--dwell` (2.5) = pause this long on a command (no taps) to confirm it.
- `--no-haptics` disables the buzz; `--backend mock` runs with no car.

Tip: tapping the **band face** (or the band against a table edge) is far easier on the wrist than flicking. First tap firm to wake the accelerometer, then light taps register.

One-time setup (already done; recorded for reproducibility)

- 1 **Tesla Fleet API app** at developer.tesla.com — Authorization Code + M2M, scopes
`vehicle_device_data vehicle_cmds vehicle_charging_cmds,origin`
`https://hamza-khan-algonquin.github.io, redirect http://localhost:3000/callback.`

1 **Signing key pair** (openssl ecparam -name prime256v1 -genkey), public key hosted at <https://hamza-khan-algonquin.github.io/.well-known/appspecific/com.tesla.3p.public-key.pem> (GitHub user-site Pages + a .nojekyll file so the dotfolder is served).

1 **Register + login + pair:**

```
python tesla_auth_PWT.py register      # registers the domain/public key
python tesla_auth_PWT.py login         # OAuth; saves tokens to secrets/
python tesla_auth_PWT.py vehicles      # gets the VIN
```

then pair the virtual key: open https://tesla.com/_ak/hamza-khan-algonquin.github.io on the phone and approve.

1 **Signing proxy:** clone github.com/teslamotors/vehicle-command, go build the tesla-http-proxy, make a localhost TLS cert.

Secrets (client secret, private key, tokens, MFA codes) are **gitignored** — never commit them.

Tool reference (src/*_PWT.py)

Tool	Purpose
whoop_protocol_PWT.py	protocol core: framing, commands, HR/accel/event/historical decode
gesture_control_PWT.py	the main app: taps → safety FSM → vehicle (+ haptics, calibrate)
vehicle_actuator_PWT.py	swappable backend: Mock / Tesla Fleet (discrete only)
tesla_auth_PWT.py	Fleet API onboarding: register / login / refresh / vehicles
safety_state_machine_PWT.py	fail-closed ARM/HOLD/STOP state machine (unit-tested)
ble_haptic_PWT.py	buzz the band's motor (test / pick a pattern)
ble_rawstream_PWT.py	enable optical + drain banked raw sensor data
decode_rawstream_PWT.py	decode a raw capture → per-second CSV
plot_rawstream_PWT.py	CSV → offline HTML biometric dashboard
gesture_PWT.py, ble_stream_PWT.py, ble_scanner_PWT.py, ...	earlier tools (streaming, scanning, events)

Safety & scope

- Vehicle control is **discrete, supervised, line-of-sight** only.
- **Fail-closed:** no command fires unless armed; any dropout → safe state; panic locks.
- **No vehicle motion by gesture** — Tesla exposes no third-party motion API, and

bypassing its Summon interlocks is out of scope. The DRIVE demo uses a simulator.

ProjWT — private portfolio project. Keep this file updated.